

Feature Selection

Sélectionner automatiquement les variables qui comptent — pour des modèles plus précis, plus rapides et plus interprétables.

● Filter

● Wrapper

● Embedded

01 Pourquoi la Feature Selection ?

Une étape critique pour les données à haute dimensionnalité

DÉFINITION

Sélectionner automatiquement un **sous-ensemble de features pertinentes et informatives** parmi toutes les features disponibles.

— OBJECTIFS

- Éliminer les features redondantes et non pertinentes
- Améliorer la précision du modèle
- Réduire le temps de calcul
- Améliorer l'interprétabilité du modèle
- Réduire le risque d'overfitting

— À NE PAS CONFONDRE

Selection On **choisit** des features originales — elles restent interprétables : « âge », « glycémie », « surface tumorale ».

Extraction On **crée** de nouvelles variables (ex. PCA) — combinaisons linéaires des features. L'interprétabilité est perdue.

— TROIS GRANDES FAMILLES

- **Filter** — évaluation statistique, indépendante du modèle
- **Wrapper** — évaluation guidée par un modèle ML
- **Embedded** — sélection intégrée à l'entraînement

02

Vue d'ensemble des trois familles

Trois stratégies, trois compromis coût / précision

FILTER

Statistique

Évaluent chaque feature indépendamment via des critères statistiques. Rapides, peu coûteuses, utilisées en prétraitement.

Variance • Pearson • Mutual Info • Chi² • ANOVA • ReliefF

WRAPPER

Guidé par modèle

Utilisent un modèle ML pour évaluer des sous-ensembles. Plus coûteuses, mais plus précises car tiennent compte des interactions.

RFE • Forward Selection • Backward Selection

EMBEDDED

Intégré

La sélection est intégrée au processus d'entraînement du modèle. Bon compromis entre coût de calcul et précision.

LASSO (L1) • Tree-Based Importance

FAMILLE	AVANTAGE	INCONVÉNIENT
Filter	Rapide, scalable	Ignore les interactions entre features
Wrapper	Tient compte des interactions	Coûteux en calcul
Embedded	Bon compromis	Dépend du modèle utilisé

03 Variance Threshold

Supprimer les features qui varient trop peu pour discriminer

FILTER

— PRINCIPE

Supprimer les features dont la variance est inférieure à un seuil défini. Une feature qui **varie peu n'apporte pas d'information discriminante**.

— FONCTIONNEMENT

- Calculer la variance de chaque feature sur le dataset
- Supprimer toute feature sous le seuil
- Paramètre clé : le seuil (défaut = 0 → features constantes)

— EXEMPLE

Dataset de 500 patients : « sexe » vaut 1 pour 498 patients. Variance quasi nulle → **supprimée**, car elle n'aide pas à discriminer les classes.

$$\text{Var}(X) = (1/n) \cdot \sum (X_i - \bar{X})^2$$

X_i	valeur de la feature pour l'observation i
$\bar{X}$	moyenne de la feature
n	nombre d'observations
→	on supprime la feature si $\text{Var}(X) < \text{seuil}$

SKLEARN

python

```
from sklearn.feature_selection import VarianceThreshold
sel = VarianceThreshold(threshold=0.01)
X_reduced = sel.fit_transform(X)
```

QUAND L'UTILISER

En **premier filtre rapide** avant toute autre méthode — notamment sur des données radiomiques ou génomiques à très haute dimensionnalité.

04 Pearson Correlation

Corrélation linéaire entre chaque feature et la cible

FILTER

— PRINCIPE

Mesurer la corrélation linéaire entre chaque feature et la variable cible. Les features **fortement corrélées à la cible** sont conservées.

— FONCTIONNEMENT

- r proche de ± 1 → forte relation linéaire → retenue
- r proche de 0 → peu informative → supprimée
- Détecte la redondance : si $|r| > 0.95$ entre deux features, n'en garder qu'une

— EXEMPLE

Prix immobilier : « surface habitable » a $r = 0.85$ → **conservée**. « Numéro de lot » a $r = 0.02$ → supprimée.

$$r(X, y) = \frac{\sum (X_i - \bar{X})(y_i - \bar{y})}{\sqrt{[\sum (X_i - \bar{X})^2 \cdot \sum (y_i - \bar{y})^2]}}$$

X_i, y_i valeurs de la feature et de la cible pour l'observation i

$\bar{X}, \bar{y}$ moyennes respectives de X et y

→ $r \in [-1, 1]$; on retient les features où $|r| >$ seuil (ex. 0.3)

PANDAS

python

```
import pandas as pd
corr = pd.DataFrame(X).corrwith(pd.Series(y))
selected = corr[abs(corr) > 0.3].index
```

Limite Détecte uniquement les relations **linéaires**. Ne capture pas les dépendances non linéaires entre feature et cible.

05 Mutual Information • SelectKBest

Information partagée entre feature et cible — linéaire ou non

FILTER

— PRINCIPE

Mesurer la quantité d'information partagée entre une feature et la cible, **qu'elle soit linéaire ou non**.

— FONCTIONNEMENT

- $I(X;Y) = 0 \rightarrow X$ et Y sont indépendants
- $I(X;Y) > 0 \rightarrow X$ apporte de l'information sur Y
- SelectKBest sélectionne les K features au score le plus élevé

— EXEMPLE

Classification de tumeurs : « texture GLCM » a une relation **non linéaire** avec le label (bénin/malin) que Pearson rate, mais que MI capture.

$$I(X ; Y) = \sum_x \sum_y p(x,y) \cdot \log \frac{p(x,y)}{p(x) \cdot p(y)}$$

$p(x,y)$ probabilité jointe de X et Y

$p(x), p(y)$ probabilités marginales de X et Y

Σ somme sur toutes les valeurs possibles de x et y

SKLEARN

python

```
from sklearn.feature_selection import SelectKBest, mutual_info_classif
sel = SelectKBest(score_func=mutual_info_classif, k=20)
X_reduced = sel.fit_transform(X, y)
```

AVANTAGE VS PEARSON

Fonctionne avec des **relations non linéaires** et des **variables catégorielles**.

06 Chi-square Test

Indépendance statistique entre feature catégorielle et cible

FILTER

— PRINCIPE

Tester l'indépendance entre une feature catégorielle et la cible. Plus le χ^2 est élevé, **plus la feature est liée à la cible**.

— FONCTIONNEMENT

- Construire un tableau de contingence (feature × cible)
- Comparer fréquences observées et attendues
- Retenir les features à χ^2 élevé (p-value faible)

— EXEMPLE

Patients malade/sain : « tabagisme » a un χ^2 élevé → **retenue**. « Groupe sanguin » a un χ^2 faible → supprimée.

$$\chi^2 = \sum_i \sum_j (O_{ij} - E_{ij})^2 / E_{ij}$$

- O_{ij} fréquence observée dans la cellule (i,j)
- E_{ij} fréquence attendue = (total ligne i × total col j) / total
- une p-value < 0.05 indique une liaison significative

SKLEARN

python

```
from sklearn.feature_selection import SelectKBest, chi2
sel = SelectKBest(score_func=chi2, k=15)
X_reduced = sel.fit_transform(X_discretized, y)
```

Attention Requiert des features **non négatives**. Ne s'applique pas directement à des valeurs continues sans discrétisation préalable.

Comparer les moyennes d'une feature numérique entre classes

— PRINCIPE

Comparer les moyennes d'une feature numérique entre les classes. Si elles **diffèrent significativement**, la feature est discriminante.

— LECTURE

- F élevé → la feature sépare bien les classes
- Variance inter-classes grande vs intra-classes faible
- Une feature aléatoire aura un F proche de 0

— EXEMPLE

Dataset Iris (3 espèces) : « longueur du pétale » a un F **très élevé** car sa moyenne diffère fortement selon l'espèce.

$$F = MS_{\text{between}} / MS_{\text{within}}$$

K	nombre de classes · N : nombre total d'observations
n_k	nombre d'observations dans la classe k
$\bar{X}_k$	moyenne de la feature dans la classe k
$\bar{X}$	moyenne globale de la feature

SKLEARN

python

```
from sklearn.feature_selection import SelectKBest, f_classif
sel = SelectKBest(score_func=f_classif, k=10)
X_reduced = sel.fit_transform(X, y)
```

Limite Suppose des distributions approximativement **normales par classe**. Sensible aux outliers.

08 ReliefF

Distinguer les classes via les plus proches voisins

FILTER

— PRINCIPE

Évaluer chaque feature selon sa capacité à **distinguer des instances de classes différentes**, en s'appuyant sur les plus proches voisins.

— FONCTIONNEMENT

- Pénalise les features qui séparent des instances de même classe
- Récompense celles qui séparent des classes différentes
- Score faible ou négatif → la feature introduit du bruit

— EXEMPLE

Cancer du sein : deux patientes aux profils MRI similaires mais de classes différentes
— une feature qui les distingue voit son score **W augmenter**.

$$W(A) \leftarrow W(A) - \text{diff}(A, x, \text{nearHit})/m + \text{diff}(A, x, \text{nearMiss})/m$$

- W(A)** score de la feature A, mis à jour à chaque itération
- nearHit** voisin le plus proche dans la même classe
- nearMiss** voisin le plus proche dans une classe différente
- m** nombre d'itérations (échantillons tirés)

SKREBATE

python

```
from skrebate import ReliefF
sel = ReliefF(n_features_to_select=20, n_neighbors=10)
sel.fit(X, y)
X_reduced = sel.transform(X)
```

AVANTAGE

Gère les **interactions entre features**, le multi-classes, et ne pose aucune hypothèse sur la distribution des données.

09 Recursive Feature Elimination

Éliminer itérativement les features les moins importantes

WRAPPER

— PRINCIPE

Entraîner un modèle, identifier les features les moins importantes, les **supprimer, et répéter** jusqu'au nombre désiré de features.

— FONCTIONNEMENT

- 1 Entraîner le modèle sur toutes les features de S
- 2 Classer les features par $|\beta_j|$
- 3 Supprimer j^* — la moins importante
- 4 Répéter jusqu'au nombre cible

$$j^* = \operatorname{argmin}_{(j \in S)} |\beta_j|$$
$$S \leftarrow S \setminus \{j^*\}$$

S	ensemble courant de features
β_j	coefficient ou score d'importance de la feature j
j^*	feature au score le plus faible → supprimée

SKLEARN

python

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
rfe = RFE(estimator=model, n_features_to_select=15)
X_reduced = rfe.fit_transform(X, y)
```

Exemple Détection de fraude (50 features) : on retient finalement les **15 features les plus influentes**.

AVANTAGE VS FILTER

Tient compte des **interactions entre features**. Combinable avec n'importe quel estimateur — SVM, Random Forest, Régression Logistique...

10 LASSO • Régularisation L1

Une pénalité qui pousse les coefficients inutiles à zéro

EMBEDDED

— PRINCIPE

Ajouter une pénalité L1 à la fonction de coût. Elle pousse certains coefficients **exactement à zéro** → sélection automatique pendant l'entraînement.

— FONCTIONNEMENT

- λ grand → plus de coefficients à zéro → moins de features
- λ optimal choisi par cross-validation

— EXEMPLE

Prédiction du diabète, 100 features cliniques : après LASSO, seules **12 features** ont un coefficient non nul (HbA1c, IMC, glycémie...). Les 88 autres sont éliminées.

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}} \left[\frac{1}{2n} \cdot \sum_i (y_i - \beta_0 - \sum_j \beta_j x_{ij})^2 + \lambda \cdot \sum_j |\beta_j| \right]$$

β_j	coefficient de la feature j · β_0 : intercept
x_{ij}	valeur de la feature j pour l'observation i
λ	paramètre de régularisation (contrôle la sparsité)

SKLEARN

python

```
from sklearn.linear_model import LassoCV
from sklearn.feature_selection import SelectFromModel
lasso = LassoCV(cv=5).fit(X, y)
sel = SelectFromModel(lasso, prefit=True)
X_reduced = sel.transform(X)
```

AVANTAGE CLÉ

Sélection **et** modélisation en une seule étape. Très efficace pour les données cliniques à haute dimensionnalité où beaucoup de features sont redondantes.

11 Tree-Based Feature Importance

Random Forest & XGBoost notent chaque feature automatiquement

EMBEDDED

— PRINCIPE

Les modèles à base d'arbres calculent un score d'importance par feature selon sa **contribution à la réduction d'impureté** à chaque nœud de séparation.

— FONCTIONNEMENT

- À chaque split, mesurer le gain d'information apporté
- Cumuler ce gain sur tous les arbres de la forêt
- Conserver les features au score le plus élevé

— EXEMPLE

Titanic : la forêt note fortement « classe du billet », « âge », « tarif » et proche de 0 « numéro de cabine » → **supprimée**.

$$\text{Importance}(A) = \sum_t [p(t) \cdot \Delta\text{Impurity}(t,A)]$$
$$\text{Gini}(t) = 1 - \sum_k p(k|t)^2$$

-
- t** nœud où A est utilisée pour le split
 - p(t)** proportion d'observations atteignant le nœud t
 - ΔImp.** réduction d'impureté obtenue grâce à A

SKLEARN

python

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
rf = RandomForestClassifier(n_estimators=100).fit(X, y)
sel = SelectFromModel(rf, prefit=True, threshold="median")
X_reduced = sel.transform(X)
```

AVANTAGE

Non linéaire, robuste aux outliers, gère les interactions entre features — et fournit une **visualisation intuitive** des features les plus utiles.

12

Le cas PCA — Extraction, pas Selection

Choisir des features existantes vs créer de nouvelles variables

EXTRACTION

IDÉE FAUSSE À CORRIGER

PCA n'est pas une technique de Feature Selection. C'est une technique de Feature Extraction (réduction de dimensionnalité).

$$Z = X \cdot W$$
$$\text{Cov}(X) \cdot W = \lambda \cdot W$$

- X
- matrice des données originales (n × p)
- W
- vecteurs propres (p × k)
- Z
- composantes principales (n × k), k < p
- λ
- valeurs propres = variance expliquée

	FEATURE SELECTION	PCA (EXTRACTION)
Principe	Sous-ensemble de features originales	Crée de nouvelles variables
Interprétab.	Conservée (« âge »...)	Perdue (PC1, PC2...)
Features	Conservées telles quelles	Transformées
Résultat	« Garder f3, f7, f12 »	PC1 = 0.32·f1 + 0.18·f2 + ...

Selection On garde des features réelles → interprétable → adapté au **domaine médical et clinique**.

PCA On projette dans un nouvel espace → utile pour la **visualisation** ou quand l'interprétabilité n'est pas requise.

13 Synthèse & guide de choix

Comparatif complet et stratégie pratique en trois étapes

MÉTHODE	FAMILLE	RELATION	COÛT
Variance Threshold	Filter	—	Très faible
Pearson	Filter	Linéaire	Faible
Mutual Information	Filter	Non linéaire	Faible
Chi-square	Filter	Catégorielle	Faible
ANOVA F-Test	Filter	Linéaire	Faible
ReliefF	Filter	Interactions	Moyen
RFE	Wrapper	Selon modèle	Élevé
LASSO	Embedded	Linéaire + sparsité	Moyen
Tree Importance	Embedded	Non linéaire	Moyen
PCA (Extraction)	—	Linéaire	Moyen

— STRATÉGIE PRATIQUE EN 3 ÉTAPES

- 1 **Filtre rapide** — Variance Threshold + Correlation : élimination des features constantes et redondantes.
- 2 **Affinage statistique** — Mutual Information ou ReliefF si des relations non linéaires sont suspectées.
- 3 **Sélection guidée par le modèle** — RFE, LASSO ou Tree Importance selon le classifieur cible.

RÈGLE D'OR

Il n'existe pas de méthode universelle. Le choix optimal dépend du **type de données**, de la nature des relations features-cible, du **modèle ML** utilisé et des contraintes d'**interprétabilité** — critère crucial en domaine médical.